AWS IAM Bulk User Management

with Groups, MFA, SSO & Terraform

Complete Project Documentation — Step-by-Step Guide + Network Flow + All Diagrams

Author	Ritik
Users Source	users.csv — 6 users across 3 departments
Departments	engineering, education, management
IAM Groups	engineering (3), education (2), management (1)
Username Format	lower(first_name[0] + last_name) e.g. rsharma
Login Profile	Auto-generated password, reset required on first login
Security	MFA enforcement + SSO via IAM Identity Center
IaC Tool	Terraform | AWS Provider | us-east-1
Date	22-05-2026

---

1. Project Overview

This project uses Terraform to bulk-provision AWS IAM users from a CSV file. Instead of creating IAM users manually one by one, a single CSV file defines all users (name, department, job title) and Terraform automatically creates all users, generates login credentials, assigns them to department-based IAM groups, and configures security policies including MFA enforcement and SSO.

- CSV-driven user creation — add a row to users.csv to create a new IAM user
- Auto-generated IAM usernames: lower(first_initial + last_name) — e.g. rsharma, akumar
- Auto-generated temporary passwords with password_reset_required = true
- Three IAM groups — engineering, education, management — membership auto-assigned by Department tag
- Account ID fetched dynamically via aws_caller_identity data source
- MFA enforced — IAM policy denies all actions if MFA is not present
- SSO configured via IAM Identity Center with permission sets per group
- Outputs: all usernames (map) + all passwords (sensitive map)

---

2. Project File Structure

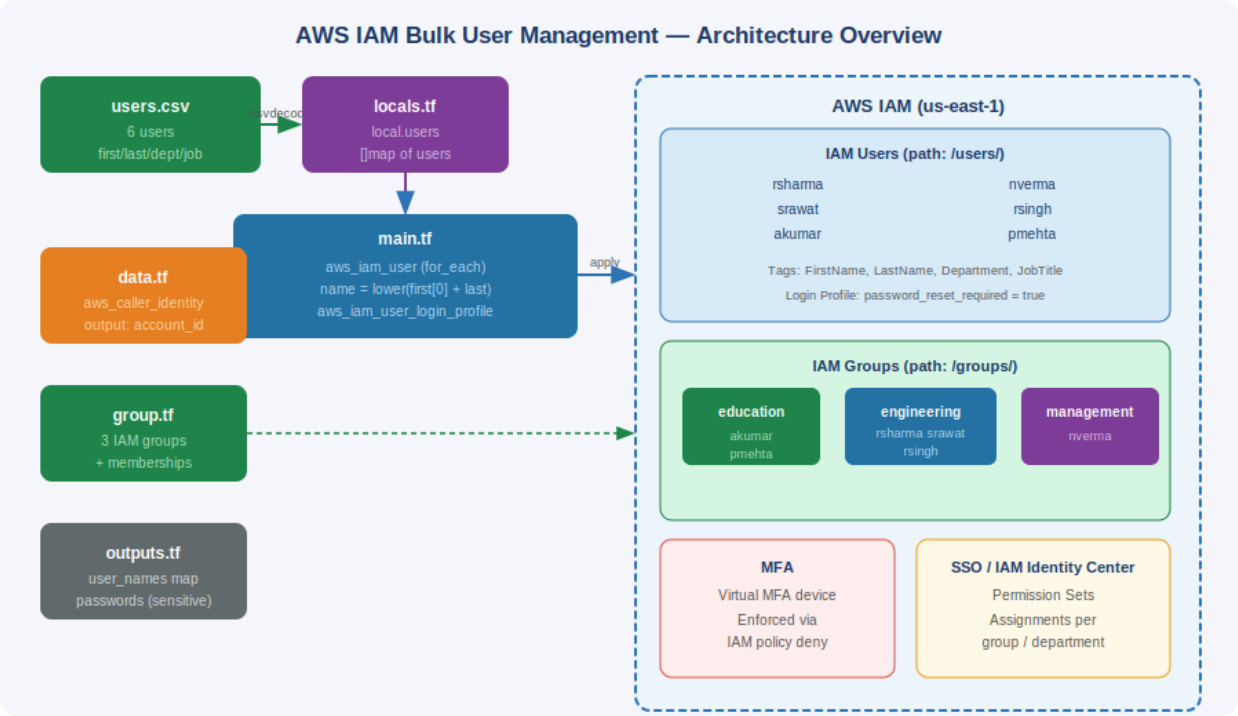
Project File Structure

```
iam-bulk-user-management/
├── users.csv       ← source of truth: 6 users with dept + job title
├── locals.tf       ← csvdecode() parses CSV into local.users list
├── data.tf         ← aws_caller_identity + output account_id
├── main.tf         ← aws_iam_user + aws_iam_user_login_profile (for_each)
├── group.tf        ← 3 IAM groups + dynamic dept-based memberships
├── outputs.tf      ← user_names map + login_passwords (sensitive)
├── variables.tf    ← password_length variable (default: 12)
├── providers.tf    ← AWS provider configuration
└── backend.tf      ← (commented) S3 remote state config
.gitignore · terraform.tfstate · terraform.tfstate.backup
```

---

3. Architecture Diagram

The diagram shows the full data flow: CSV → locals → Terraform resources → AWS IAM (users, groups, login profiles, MFA, SSO).



---

4. Username Generation Logic

IAM usernames are derived automatically from the CSV using Terraform's string functions. The formula takes the first character of the first name, appends the full last name, and converts everything to lowercase.

IAM Username Generation Logic

Formula: lower(substr(first_name, 0, 1) + last_name)

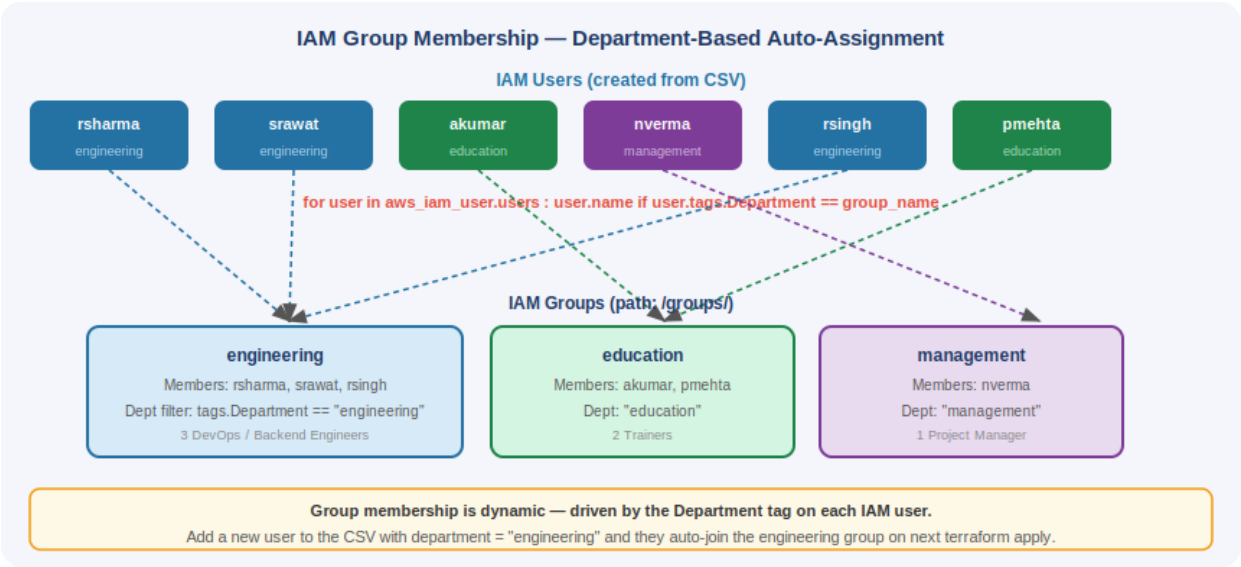
Takes first character of first_name, appends full last_name, converts entire string to lowercase

first_name	last_name	department	job_title	IAM Username
Ritik	Sharma	engineering	DevOps Engineer	rsharma
Sonali	Rawat	engineering	DevOps Engineer	srawat
Amit	Kumar	education	Trainer	akumar
Neha	Verma	management	Project Manager	nverma
Rahul	Singh	engineering	Backend Engineer	rsingh
Priya	Mehta	education	Cloud Trainer	pmehta

---

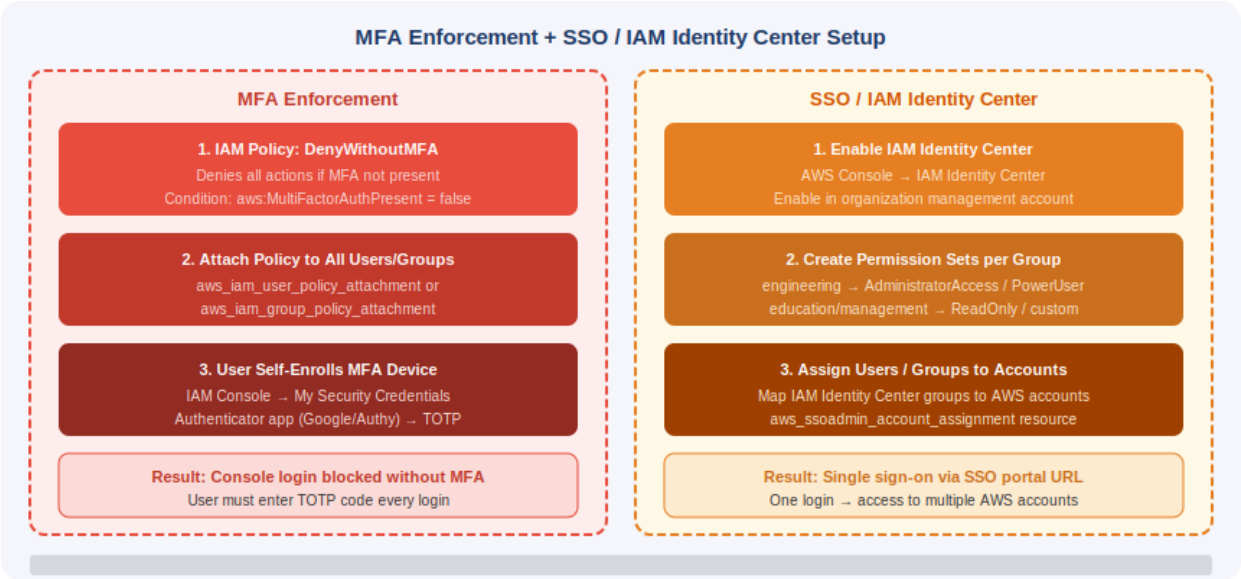
5. IAM Group Membership Flow

Group membership is driven by the Department tag on each IAM user. Terraform uses a for expression with an if filter to build the members list per group dynamically — no hardcoding of usernames.



---

6. MFA Enforcement & SSO Setup



---

7. Resources — What Each Does & Why

Resource	What It Does	Why It Is Needed
`users.csv`	CSV file with columns: first_name, last_name, department, job_title	Single source of truth for all users. Adding a row = new IAM user on next terraform apply. No code changes required.
`locals.tf csvdecode()`	Parses the CSV into a list of maps (local.users)	csvdecode() converts the raw CSV string into structured data Terraform can iterate over with for_each. Without it, you'd have to hardcode each user in .tf files.
`data.aws_caller_identity`	Fetches the AWS Account ID of the current caller	Lets you reference the account ID dynamically (e.g., in ARNs or conditions) without hardcoding a numeric account ID that could change.
`aws_iam_user (for_each)`	Creates one IAM user per row in the CSV	for_each loops over local.users, generating a unique IAM user per row. The key is lower(first[0]+last) — used as both map key and username.
`substr() + lower()`	Builds username: lower(first_name[0] + last_name)	Generates consistent, predictable usernames without manual input. substr(str,0,1) extracts the first character only.
`path = /users/`	Organises users under a logical IAM path namespace	IAM paths group resources logically (like folders). /users/ separates IAM users from groups, roles, and service accounts.
`tags on aws_iam_user`	Stores FirstName, LastName, Department, JobTitle as tags	Tags are metadata — used later in group membership filters (user.tags.Department) and for auditing/cost tracking.
`aws_iam_user_login_profile`	Creates a console login	Without a login profile, IAM users cannot log into

	password for each user	the AWS Console. password_reset_required = true forces a password change on first login.
`lifecycle ignore_changes`	Ignores changes to password_length after creation	Prevents Terraform from regenerating passwords on every apply (which would lock existing users out). Only the initial password creation matters.
`output login_passwords`	Outputs all generated passwords as a sensitive map	Lets the admin retrieve initial passwords after terraform apply to distribute to users. sensitive = true hides them from plain-text logs.
`aws_iam_group (×3)`	Creates 3 IAM groups: education, engineering, management	Groups allow attaching policies once to many users. Managing permissions on groups (not individual users) is best practice.
`aws_iam_group_membership`	Adds users to groups based on their Department tag	Uses a dynamic for expression with if filter — users are auto-sorted into groups based on department tag without any manual list management.
`MFA IAM policy`	Denies all actions if MFA is not authenticated	Prevents compromised passwords from being exploited — attacker also needs the TOTP device. Required for security compliance (CIS, SOC2, ISO27001).
`IAM Identity Center (SSO)`	Provides single sign-on across AWS accounts per group	Users get one set of credentials to access multiple AWS accounts with correct permissions per group. Eliminates per-account IAM user management.

---

8. Step-by-Step Implementation Procedure

Step 1 — Create users.csv (Source of Truth)

A CSV file was created with four columns: first_name, last_name, department, job_title. This is the only file that needs to be updated when a new user is added — no Terraform code changes needed.

```
first_name,last_name,department,job_title
Ritik,Sharma,engineering,DevOps Engineer
Sonali,Rawat,engineering,DevOps Engineer
Amit,Kumar,education,Trainer
Neha,Verma,management,Project Manager
Rahul,Singh,engineering,Backend Engineer
Priya,Mehta,education,Cloud Trainer
```

Step 2 — Fetch AWS Account ID (data.tf)

A data source was added to dynamically fetch the AWS account ID of the current caller. The account ID is output directly from data.tf so it is available for reference in policies, ARN conditions, or audit logs.

```
data "aws_caller_identity" "current" {}


output "account_id" {
  value = data.aws_caller_identity.current.account_id
```

```
}
```

Step 3 — Parse CSV into Local Variable (locals.tf)

The csvdecode() built-in function reads the CSV file from disk at plan time and converts it into a list of maps — one map per row, with column names as keys. This becomes local.users, which all other resources iterate over.

```
locals {
  users = csvdecode(file("${path.module}/users.csv"))
}
```

After this, local.users is a list like: [{first_name="Ritik", last_name="Sharma", department="engineering", job_title="DevOps Engineer"}, ...]

Step 4 — Output All Usernames (outputs.tf)

An output variable was added to display all created IAM usernames as a key→value map after terraform apply. This confirms which username was generated for each user.

```
output "user_names" {
  value = {
    for key, value in aws_iam_user.users :
    key => value.name
  }
}
```

Step 5 — Create IAM Users with Username Formula (main.tf)

IAM users are provisioned using for_each over local.users. The key (and username) is built using substr() + string interpolation + lower(). This creates consistent, lowercase usernames like rsharma, srawat, akumar. Each user is tagged with all four CSV fields.

```
resource "aws_iam_user" "users" {
  for_each = {
    for user in local.users :
    lower("${substr(user.first_name, 0, 1)}${user.last_name}") => user
  }

  name = lower("${substr(each.value.first_name, 0, 1)}${each.value.last_name}")
  path = "/users/"

  tags = {
    FirstName  = each.value.first_name
    LastName   = each.value.last_name
    Department = each.value.department
    JobTitle   = each.value.job_title
  }
}
```

Step 6 — Create Login Profiles + Output Passwords

aws_iam_user_login_profile creates a console login password for each IAM user. password_reset_required = true forces users to change their password on first login. The lifecycle block prevents Terraform from regenerating passwords on subsequent applies (which would lock users out).

```
resource "aws_iam_user_login_profile" "users" {
  for_each = aws_iam_user.users

  user                    = each.value.name
  password_reset_required = true

  lifecycle {
    ignore_changes = [password_length]
  }
}

# In outputs.tf:
output "login_passwords" {
  sensitive = true
  value = {
    for key, value in aws_iam_user_login_profile.users :
    key => value.password
  }
}
```

Step 7 — Create IAM Groups & Memberships (group.tf)

Three IAM groups were created: education, engineering, and management — each under path /groups/. Group memberships use a dynamic for expression with an if condition filtering by the Department tag, so users are auto-sorted into the correct group based on their CSV department value. No hardcoded username lists.

```
resource "aws_iam_group" "education"   { name = "education"   path = "/groups/" }
resource "aws_iam_group" "engineering" { name = "engineering" path = "/groups/" }
resource "aws_iam_group" "management"  { name = "management"  path = "/groups/" }

resource "aws_iam_group_membership" "engineering_members" {
  name = "engineering-membership"
  users = [
    for user_key, user in aws_iam_user.users : user.name
    if user.tags.Department == "engineering"
  ]
  group = aws_iam_group.engineering.name
}
# Same pattern for education_members and management_members
```

Step 8 — Enable MFA for All Users

MFA is enforced by creating an IAM policy that denies all AWS actions if the request is not authenticated with MFA. This policy is attached to all users (or to each group). Users must enroll a virtual MFA device (authenticator app) via the IAM console before they can perform any actions.

```
# IAM Policy: Deny everything unless MFA is present
```

```
resource "aws_iam_policy" "enforce_mfa" {
  name = "EnforceMFA"
  policy = jsonencode({
    Statement = [{
      Effect = "Deny"
      Action = "*"
      Resource = "*"
      Condition = {
        BoolIfExists = {
          "aws:MultiFactorAuthPresent" = "false"
        }
      }
    }]
  })
}

# Attach to each group
resource "aws_iam_group_policy_attachment" "mfa_engineering" {
  group      = aws_iam_group.engineering.name
  policy_arn = aws_iam_policy.enforce_mfa.arn
}
```

Step 9 — Enable SSO via IAM Identity Center

SSO was configured using AWS IAM Identity Center (formerly AWS SSO). Permission sets define what each group can do. Account assignments map groups to AWS accounts with the appropriate permission set. Users log in once via the SSO portal and access all assigned accounts.

```
# Enable IAM Identity Center (done via AWS Console or Organizations)

# Create permission set for engineering
resource "aws_ssoadmin_permission_set" "engineering_ps" {
  name             = "EngineeringAccess"
  instance_arn     = local.sso_instance_arn
  session_duration = "PT8H"
}

# Assign engineering group to AWS account with permission set
resource "aws_ssoadmin_account_assignment" "engineering" {
  instance_arn       = local.sso_instance_arn
  permission_set_arn = aws_ssoadmin_permission_set.engineering_ps.arn
  principal_id       = aws_identitystore_group.engineering.group_id
  principal_type     = "GROUP"
  target_id          = data.aws_caller_identity.current.account_id
  target_type        = "AWS_ACCOUNT"
}
```

Step 10 — Deploy with Terraform

```
terraform init      # Download AWS provider
```

```
terraform validate  # Check syntax
terraform plan      # Preview: 20+ resources to create
terraform apply     # Create all IAM users, groups, profiles
terraform output login_passwords  # Retrieve initial passwords
```

---

9. Variables & Outputs Reference

Name	Type / Default	Purpose
`password_length`	number / 12	Length of auto-generated IAM login passwords
`local.users`	list(map) / CSV	Parsed list of all users from users.csv via csvdecode()
`output: account_id`	string	AWS account ID from aws_caller_identity data source
`output: user_names`	map(string)	Map of username key → IAM username for all created users
`output: login_passwords`	map(string) sensitive	Map of username key → initial login password (hidden in logs)

---

10. Project Summary — Step by Step

Everything done in this project — concise numbered summary

1. Created users.csv with columns first_name, last_name, department, job_title — containing 6 users across engineering, education, and management departments.
2. Added data.aws_caller_identity in data.tf to dynamically fetch the AWS account ID, and output it as account_id — no hardcoded account numbers.
3. Created locals.tf using csvdecode(file(...)) to parse the CSV into local.users — a list of maps Terraform can iterate over with for_each.
4. Added output user_names in outputs.tf using a for expression over aws_iam_user.users to display all created IAM usernames as a map after apply.
5. Created aws_iam_user resources in main.tf using for_each, generating each username with lower(substr(first_name, 0, 1) + last_name). Each user has path = /users/ and four tags: FirstName, LastName, Department, JobTitle.
6. Created aws_iam_user_login_profile for every user with password_reset_required = true and a lifecycle block ignoring password_length changes. Output login_passwords (sensitive) prints all passwords after apply.
7. Created group.tf with three aws_iam_group resources (education, engineering, management under path /groups/) and three aws_iam_group_membership resources using dynamic for-if expressions to auto-assign users based on their Department tag.
8. Configured MFA enforcement: created an IAM policy (DenyWithoutMFA) that denies all AWS actions when aws:MultiFactorAuthPresent = false, and attached it to all groups. Users must self-enroll a virtual MFA device on first login.

9. Configured SSO via AWS IAM Identity Center: created permission sets per group (e.g. EngineeringAccess), assigned groups to the AWS account, and enabled the SSO portal so users log in once to access all assigned accounts.
10. Deployed with terraform init → validate → plan → apply. All users, groups, memberships, login profiles, and security policies were created automatically from the CSV.

Result: 6 IAM users auto-created from a CSV, sorted into 3 department groups, each with a temporary login password, MFA enforced, and SSO access configured — all provisioned with a single terraform apply.